

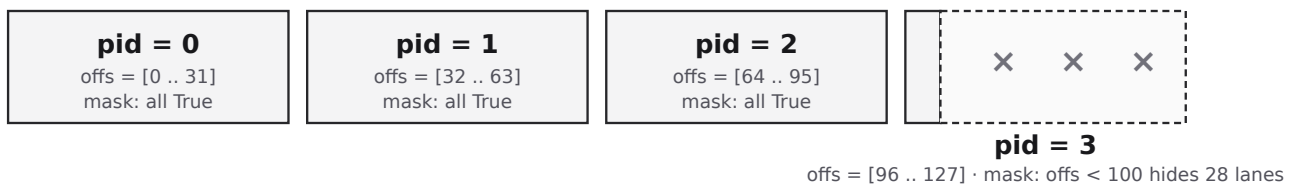
Triton DSL Cheatsheet

Triton lets you write GPU kernels in Python. Here's the model, the primitives, and the gotchas — roughly in the order you hit them when writing a kernel. Full API at triton-lang.org · companion to “Write Your First High-Performance GPU Kernel in Python!” · `import triton; import triton.language as tl`.

§1 · The mental model

You launch a [grid of programs](#); each owns one `BLOCK`-sized chunk; the last one masks its tail.

$$n = 100 \cdot \text{BLOCK} = 32 \cdot \text{grid} = (\text{cdiv}(100, 32),) = (4,)$$



```
grid = (triton.cdiv(n, BLOCK),)      # one program per chunk
kernel[grid](a, b, out, n, BLOCK=1024)
```

If `n` isn't a multiple of `BLOCK`, you have a tail — mask every load and store.

§2 · Kernel anatomy and launch

`@triton.jit` captures source; the first launch compiles; the second is a dict lookup.

```
@triton.jit
def kernel(a_ptr, b_ptr, out_ptr, n, BLOCK: tl.constexpr):
    pid = tl.program_id(axis=0)
    offs = pid * BLOCK + tl.arange(0, BLOCK)      # BLOCK must be a power of two
    mask = offs < n                               # tail-safe
    a = tl.load(a_ptr + offs, mask=mask)
    b = tl.load(b_ptr + offs, mask=mask)
    tl.store(out_ptr + offs, a + b, mask=mask)

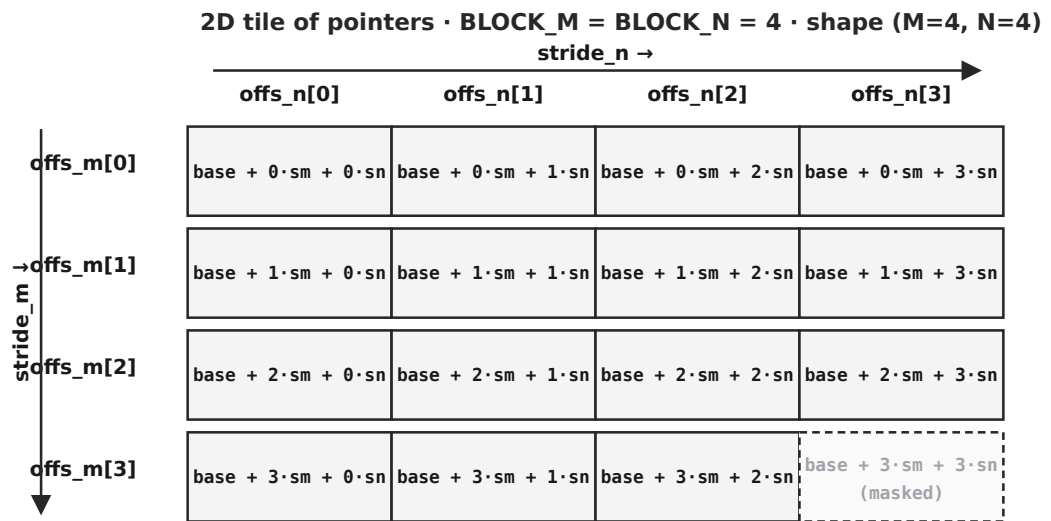
compiled = kernel[grid](a, b, out, n, BLOCK=1024)  # returns a CompiledKernel
```

`grid` may also be `lambda meta: (triton.cdiv(n, meta['BLOCK']),)` — useful with autotune. Inspect IR via `compiled.asm['ttir' | 'ttgir' | 'llir' | 'ptx']`. If the natural size isn't a power of two, round up with `triton.next_power_of_2`.

Change a `tl.constexpr` value (e.g. `BLOCK`) and you trigger a fresh compile.

§3 · 2D tiles: broadcasting pointers

A 2D tile is a column vector times a row vector — pointers, not data.



$\text{sm} = \text{stride_m}$ · $\text{sn} = \text{stride_n}$ · ptr math is typed: + offs advances offs · dtype.itemsize bytes

```
offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)      # (M,)
offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)      # (N,)
mask    = (offs_m[:, None] < M) & (offs_n[None, :] < N)
ptrs    = base + offs_m[:, None] * stride_m + offs_n[None, :] * stride_n
```

`[ :, None ]` / `[ None, : ]` broadcast like NumPy.

`ptr + offs` advances `offs · dtype.itemsize` bytes, not `offs` bytes.

§4 · Load, store, compute

Loads read masked positions and fill the misses with `other`; stores write masked positions and skip the misses.

`tl.load(a_ptr + offs, mask=mask, other=0.0) · n = 6, BLOCK = 8`

ptr = a_ptr+0 a_ptr+1 a_ptr+2 a_ptr+3 a_ptr+4 a_ptr+5 a_ptr+6 a_ptr+7

0.42	1.07	3.14	2.71	0.05	9.99	past n	past n
------	------	------	------	------	------	--------	--------

mask = offs < n ✓ ✓ ✓ ✓ ✓ ✗ ✗

`[0.42, 1.07, 3.14, 2.71, 0.05, 9.99, 0.0, 0.0]`

loaded vector · masked lanes filled with other=0.0

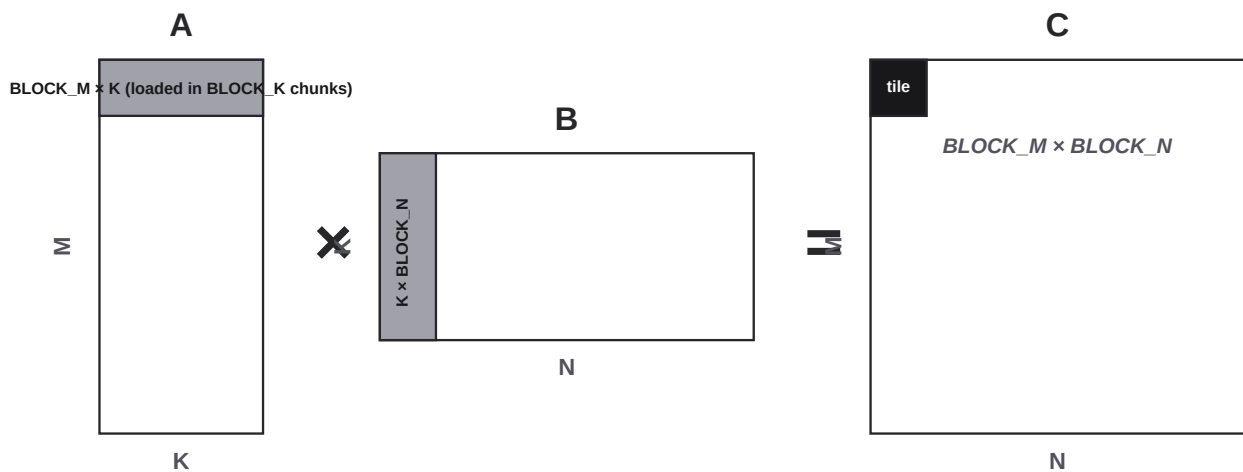
```
x = tl.load(ptr + offs, mask=mask, other=0.0)      # masked lanes return `other`
tl.store(ptr + offs, value, mask=mask)
```

Elementwise (broadcast like NumPy): `+`, `-`, `*`, `/`, `**`, `tl.exp`, `tl.log`, `tl.sqrt`, `tl.rsqrt`, `tl.sigmoid`, `tl.abs`, `tl.maximum`, `tl.minimum`, `tl.where(cond, a, b)`. Reductions (collapse one axis): `tl.sum`, `tl.max`, `tl.min`, `tl.argmax`, `tl.cumsum`. For [strided](#)/repeated 2D access: `tl.make_block_ptr` + `tl.advance`.

Forgetting `mask=` past the tail is the #1 way to corrupt memory.

§5 • `tl.dot` — tensor cores in a tile

One program computes one `BLOCK_M × BLOCK_N` tile of C by sweeping K with the accumulator in registers.



One program computes one tile of C · K-loop iterates in BLOCK_K chunks · accumulator (acc) stays in registers

```
acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
for k in range(0, K, BLOCK_K):
    a_tile = tl.load(a_ptrs)           # (BLOCK_M, BLOCK_K)
    b_tile = tl.load(b_ptrs)           # (BLOCK_K, BLOCK_N)
    acc = tl.dot(a_tile, b_tile, acc=acc) # FP32 accumulator
    a_ptrs += BLOCK_K * stride_ak
    b_ptrs += BLOCK_K * stride_bk
```

[Tensor cores](#) need FP16/BF16 inputs and `M, N, K ≥ 16` — otherwise you fall back to FP32 FMA.

§6 • What `@triton.jit` actually compiles

Your [Python](#) becomes Triton-IR → TritonGPU-IR → LLVM IR → PTX → SASS — cached by specialization key.



Cached in `~/.triton/cache/` (override with `TRITON_CACHE_DIR`). Cache key includes dtypes + every `tl.constexpr` value + pointer alignment hints — change any one, fresh compile.

Source edit not taking effect? `rm -rf ~/.triton/cache/`.

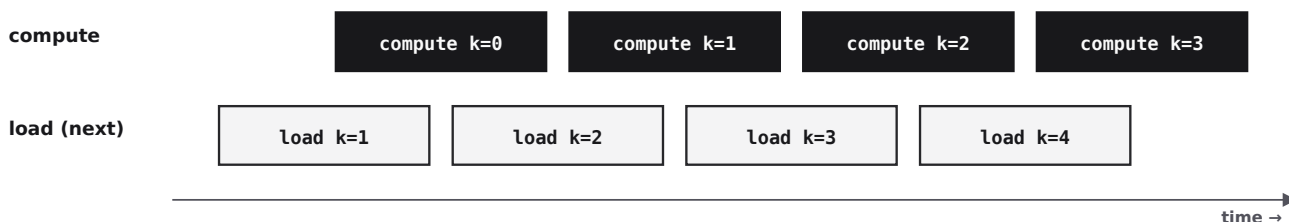
§7 · Control flow, types, atomics

Python `if` is for compile-time scalars; per-lane decisions use `tl.where`.

```
y = tl.where(x > 0, x, 0.0)          # per-lane choice
for k in tl.static_range(0, NUM_TILES): # unrolled at compile time
    ...
```

Dtypes: `tl.float32` / `float16` / `bfloat16` / `int32` / `int64` / `float8_e4m3` / `float8_e5m2` (FP8 needs `sm_90+`). Cast explicitly: `x.to(tl.float32)`. Atomics for cross-program writes (histograms, scatter): `tl.atomic_add(ptr + offs, value, mask=mask)`, also `tl.atomic_max`, `tl.atomic_min`, `tl.atomic_xchg`, `tl.atomic_cas`. Slow; prefer per-block reduction + one writeback.

`num_stages = 2` · one tile loads while the previous tile computes



`num_stages = 2` overlaps the next tile's load with the current tile's compute (see §8 to tune it).

`tl.static_range` unrolls and bloats the kernel — keep it under ~16 iterations.

§8 · Autotune

Compile every config, benchmark on the first call per `key=`, cache the winner.

First call with new `key=(M, N, K)` → compile each config (parallel) → benchmark → cache the winner.

config	compile	benchmark	measured
BLOCK = (64, 64, 32) num_warps = 4	<button>compile</button>	<button>benchmark</button>	2.10 μ s
BLOCK = (128, 128, 32) num_warps = 8	<button>compile</button>	<button>benchmark</button>	1.42 μ s ← winner
BLOCK = (128, 64, 64) num_warps = 4	<button>compile</button>	<button>benchmark</button>	1.78 μ s

→ **cache[key]**

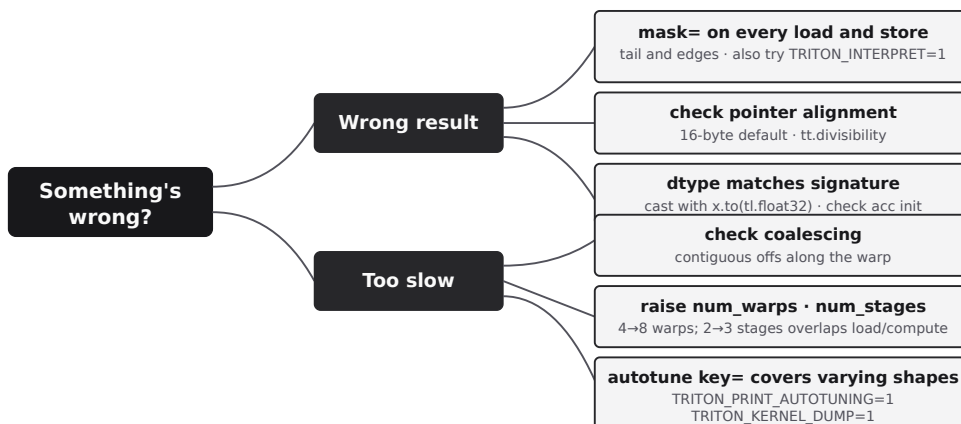
```
@triton.autotune(
    configs=[
        triton.Config({'BLOCK_M': 64, 'BLOCK_N': 64, 'BLOCK_K': 32}, num_warps=4),
        triton.Config({'BLOCK_M': 128, 'BLOCK_N': 128, 'BLOCK_K': 32}, num_warps=8),
        triton.Config({'BLOCK_M': 128, 'BLOCK_N': 64, 'BLOCK_K': 64}, num_warps=4),
    ],
    key=['M', 'N', 'K'],          # re-tune when these change
)
@triton.jit
def matmul_kernel(...): ...
```

`num_warps` (1–16): warps per program — more = more parallelism, more register pressure. `num_stages` (2–4): software-pipeline depth (see §7). For configs computed from runtime args (instead of benchmarked), use `triton.heuristics({'BLOCK': lambda meta: triton.next_power_of_2(meta['N'])})`.

Without `key=`, the first winner sticks forever — even when `M`, `N`, `K` change.

§9 · Debugging and gotchas

Symptom decides the knob. Helpers worth knowing: `tl.static_assert(BLOCK >= 16)` for compile-time checks; `tl.device_print("pid", pid, "x", x)` to log from the kernel (slow, strip before shipping).



Always true: power-of-two tiles · always mask the tail · no `print`, no allocations, no recursion inside a kernel · pointers default to 16-byte alignment.

§10 · Go deeper

The cheatsheet shows you *what* to write; these explainers on abhik.ai/concepts cover *why* the hardware and compiler behave this way. Read one when a primitive above still feels like a black box.

- [GPU Streaming Multiprocessor \(SM\)](#) — §1–2. “A grid of programs” is the abstraction; the SM is the physical core that runs your blocks, holds their registers, and caps occupancy. The reason `num_warps` and tile size trade off against each other.

- [GPU Memory Hierarchy](#) — §1, §5. Registers → shared memory → L2 → HBM, and the latency cliff between each tier. Tiling exists to climb data up this hierarchy once and reuse it.
- [High Bandwidth Memory \(HBM\)](#) — §4. Every uncached `tl.load` / `tl.store` is an HBM transaction. This is the bandwidth ceiling your kernel is measured against — and why contiguous, coalesced access wins.
- [Memory Access Patterns: Sequential vs Strided](#) — §3–4. Why the stride in your pointer arithmetic decides whether a load is one coalesced transaction or many — a 10×-plus swing in effective bandwidth.
- [Tensor Cores](#) — §5. The matrix-multiply hardware `tl.dot` targets: what FP16/BF16 inputs buy you, why `M, N, K ≥ 16`, and the FP32-FMA fallback when you miss those rules.
- [Python Bytecode Compilation](#) — §6. How Python source becomes something executable at all — the mental model that makes Triton's Python → IR → PTX → SASS pipeline click.